Performance Evaluation of TinyLlama: Effects of Temperature and Concurrency

1 Introduction

In this report, I evaluate whether a tiny language model can run usefully on an embedded, low-power device such as the `BEAGLE-AI64`, and how its performance changes under different operating conditions. The system was able to serve responses locally, reaching about 0.3–0.4 responses per second with a tiny model (`TinyLlama`), showing that practical LLM deployment on constrained embedded hardware is feasible. Building on this, I then tested the system in a server-like setting, where an increasing number of concurrent users interact with the model, in order to evaluate how the device behaves under load.

The main focus of the report is the effect of temperature on response behavior, especially response time and generated output length. User concurrency is included as a dimension to distinguish temperature-related effects from queueing effects. This makes the study relevant not only for embedded systems, but also for LLM performance evaluation more generally, since temperature directly affects the model's generation behavior. The results show that higher temperatures mainly increase the number of generated tokens, while the generation time per token remains almost constant. In particular, increasing the temperature from 0.0 to 1.2 can change overall performance by up to 30%. Concurrency has an even stronger effect on latency, since additional users mostly create queueing rather than proportional throughput gains.

2 Performance Model

The benchmark is modelled as a closed workload. A fixed number of users, named N, repeatedly sends prompts to the local LLM server. In each round, all users submit one request. Once the responses for the round are completed, the next prompt is submitted. The think time is therefore set to $Z = 0$. This is a closed workload because the number of active users is fixed and controlled by the experiment. The benchmark does not impose a predefined external arrival rate; instead, the request rate emerges from the interaction between the fixed user population and the system response time. The arrivals are bursty, since all users submit their prompt at the beginning of each round, but the workload remains closed because the population of users is constant throughout each experiment.

For a closed system, the relationship between the number of users N, the throughput X, the response time R, and the think time Z is:

$$N = X(R + Z). \tag{1}$$

Since the experiments use zero think time, this becomes:

$$R = \frac{N}{X}. \tag{2}$$

This relation is useful for interpreting the concurrency results. If the number of users increases but throughput does not increase proportionally, the response time must increase. In queueing terms, this indicates that additional requests are waiting for a limited service resource.

In our setup, Ollama was used with its default request-level parallelism configuration. Therefore, for a given loaded model, the server effectively processed one generation request at a time, while simultaneous requests waited in a queue. From the point of view of request execution, the system can therefore be approximated as a single-server queue. This

Metric	Meaning
Response time	End-to-end user-perceived latency.
Output tokens	Number of tokens generated by the model.
Time per token	Generation duration divided by output tokens.
Response throughput	Completed responses per second.
Token throughput	Generated tokens per second.

Table 1. Different metrics we used to measure performance.

assumption is important for interpreting the results: increasing the number of users *should* mainly increase queueing delay rather than providing proportional throughput improvement.

3 Metrics

Table 1 summarizes the metrics used in the evaluation. With these metrics we try to isolate the different phenomena causing performance variation. Changing the temperature may change the number of tokens generated for each answer, impacting the response time and thus the throughput. At the same time, if response time increases without an increase in output length, then the cause may be queueing, decoding inefficiency, or other system-level overhead.

4 Experimental Setup

The experiments were conducted on a `BEAGLE-AI64` board running Ubuntu 22.04. `Ollama` was used as the inference engine to serve the `TinyLlama` model. The prompts were selected from the Dolly 15k instruction dataset. The following temperature values were evaluated: [0.0, 0.2, 0.4, 0.6, 0.8, 1.0, 1.2]

The number of concurrent users was varied from 1 to 6. For each configuration, every user submitted 60 requests. Therefore, each setup produced a total number of responses equal to 60 multiplied by the number of users.

To ensure that the prompts did not bias the results, all users submitted the prompts in the same order. This made it possible to compare the different setups more fairly, since any observed differences were less likely to be caused by variations in prompt length or difficulty.

The `TinyLlama` model was configured to generate a maximum of 250 output tokens per request. This limit was introduced to prevent unusually long responses from disrupting the experiments. During preliminary testing, some prompts produced outputs exceeding 10,000 tokens, which significantly slowed down the experiments and, in some cases, caused them to fail.

5 Results

5.1 Temperature-Centered Analysis

Figure 1 shows the mean median response time as a function of temperature, averaged across user counts. The error bars represent the standard deviation across user counts. Response time remains relatively stable up to approximately $\theta = 0.8$. At higher temperatures, especially $\theta = 1.0$ and $\theta = 1.2$, response time increases. This suggests that high temperatures create additional work for the system, but the effect becomes visible mainly at the upper end of the tested range.

Figure 2 shows the mean number of generated output tokens versus temperature. The token count increases more clearly than the response time. Low and medium temperatures produce similar output lengths, while temperatures 1.0 and 1.2 produce noticeably longer answers. This indicates that temperature mainly affects the amount of generated text.

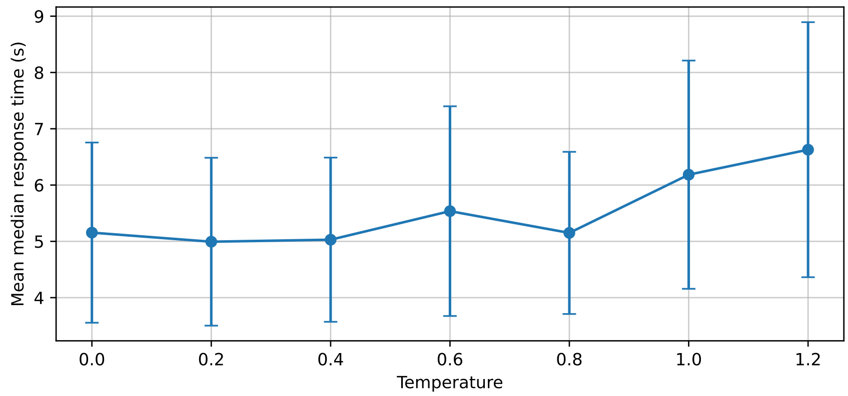

Fig. 1. Mean median response time vs temperature, averaged across user counts. Error bars show variance.

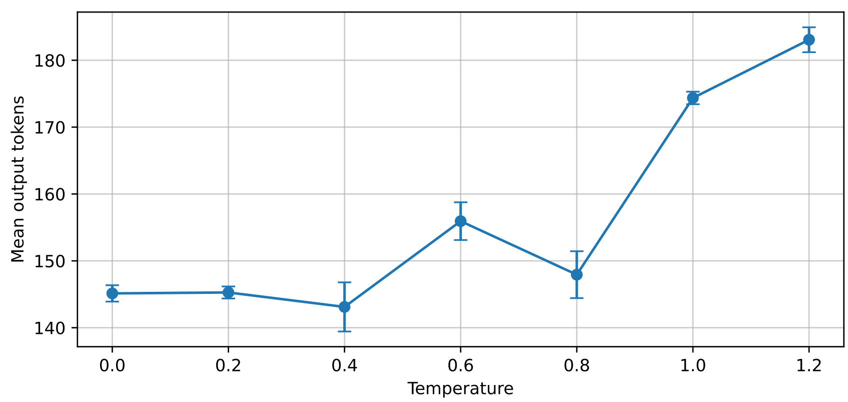

Fig. 2. Mean output tokens versus temperature, averaged across user counts. Error bars show standard deviation across user counts.

Figure 3 shows the generation time per token. The generation time per token is almost constant across temperatures. Therefore, changing temperature does not significantly change the cost of generating each token. The increase in total response time at high temperature is mainly explained by longer responses.

5.2 User-Centered Analysis

Concurrency is another factor that affects server load and response latency. Figure 4 shows how response time changes as the number of concurrent users increases, across different temperature settings.

The results show that response time increases noticeably as the number of users grows. This behavior is expected when the server becomes busy and cannot process all incoming requests simultaneously. In our setup, Ollama can handle only one user request at a time. Therefore, when multiple users submit requests concurrently, Ollama processes one request while the remaining requests wait in a queue. As the number of users increases, the queueing delay also becomes larger, which leads to longer response times. This shows that concurrency and the number of users play

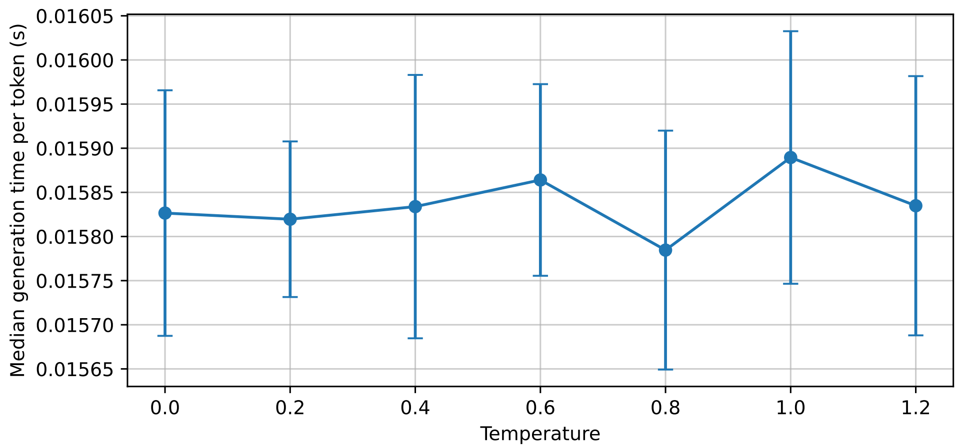

Fig. 3. Median generation time per token versus temperature, averaged across user counts.

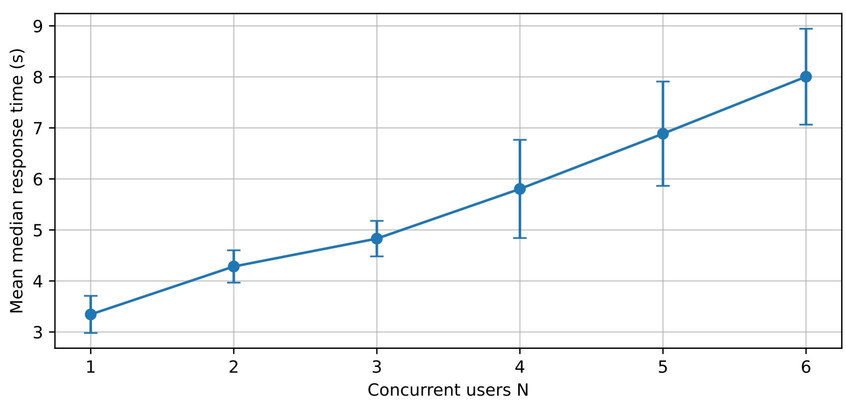

Fig. 4. Mean median response time versus number of users, averaged across temperatures.

an important role in system performance. In this experiment, the increase in response time is mainly caused by the growing queueing delay introduced by concurrent requestsr.

Figure 5 shows how response throughput changes depending on the number of active users. Interestingly, throughput also increases slightly as the number of users grows. This does not contradict the fact that Ollama processes one request at a time. We did not study the reason behind this phenomena, but a possibility is that with a single user, the server may experience small idle intervals between consecutive requests due to client-side scheduling. With multiple users, several requests are already waiting in the queue, so the server can start the next request immediately after completing the previous one. In other words, increasing the number of users may improve server utilization. However, the throughput gain is modest compared with the increase in response time. This indicates that the system is approaching the saturation throughput of the bottleneck resource. Once the server is almost continuously busy, adding more users cannot produce proportional throughput improvement; it mainly increases waiting time.

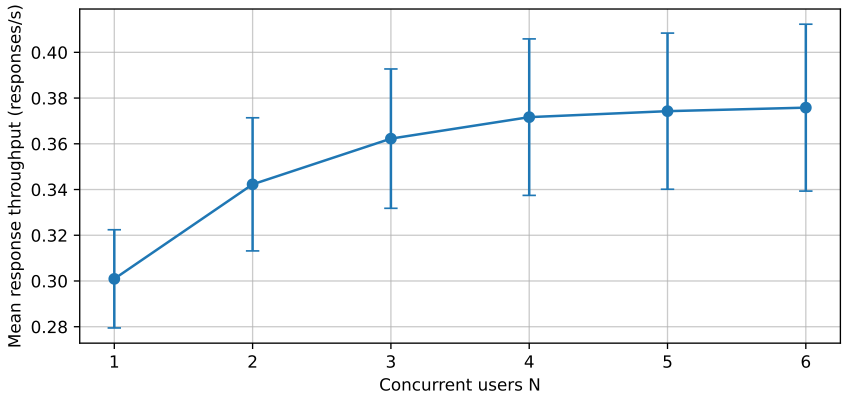

Fig. 5. Mean response throughput versus number of users, averaged across temperatures.

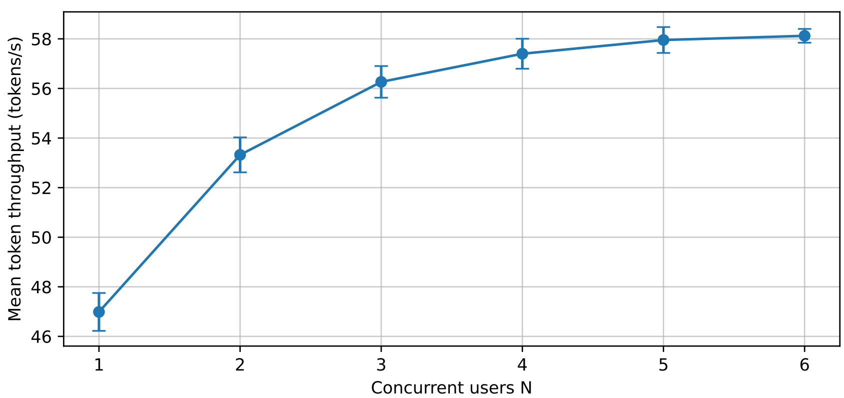

Fig. 6. Mean token throughput versus number of users, averaged across temperatures.

Token throughput behaves similarly to response throughput. It increases slightly with more users because the server is kept busy for a larger fraction of the experiment. Nevertheless, the improvement is much smaller than the increase in response time. Therefore, adding users increases utilization up to the bottleneck limit, but after that point it mostly increases queueing delay rather than useful completed work.

5.3 Interaction Between Temperature and Concurrency

The aggregated plots are useful, but they hide interactions between temperature and concurrency. Figure 7 shows median response time versus temperature with one curve per user count. The curves are mainly separated by the number of users, meaning that concurrency is the dominant factor for response time. However, high temperatures increase latency more clearly when there are more users. Longer responses occupy the server for longer, which increases the queueing delay of other users.

Figure 8 shows response throughput versus temperature for each user count. Throughput tends to decrease slightly at higher temperatures, especially where output length increases. This is expected: if each response contains more tokens, each request requires more service time, and fewer responses can be completed per second.

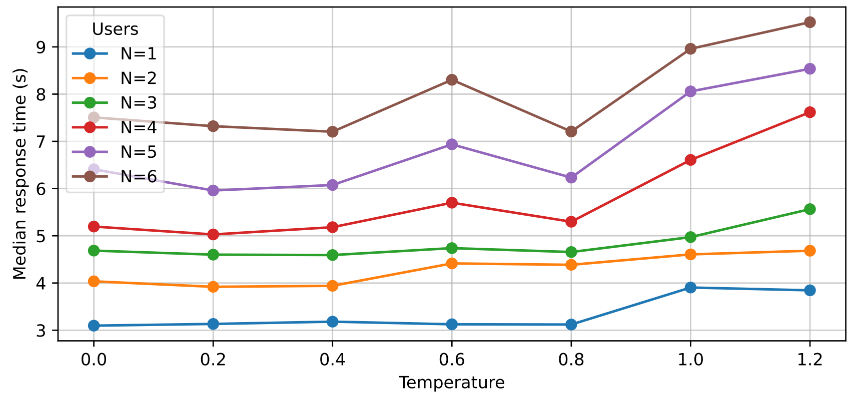

Fig. 7. Median response time versus temperature for each number of users.

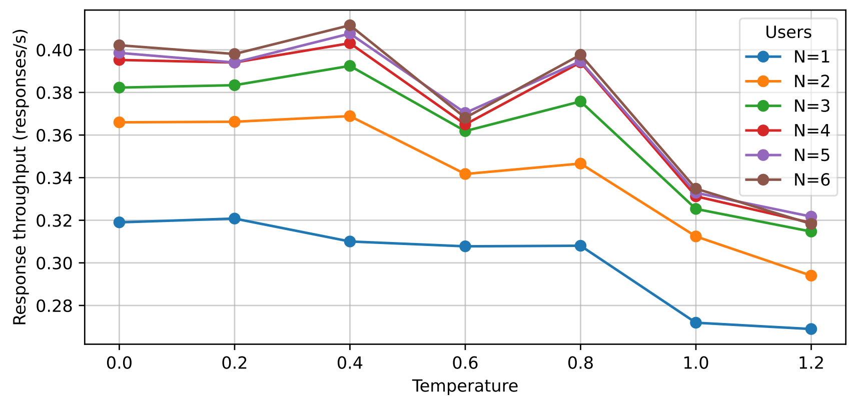

Fig. 8. Response throughput versus temperature for each number of users.

The curves are close to each other, Figure 9 shows output tokens versus temperature for each user count. showing that concurrency does not strongly affect output length. Output length is controlled mainly by temperature.

Figure 10 shows generation time per token. This figure confirms that per-token generation speed is stable. The main latency increase at high temperature is explained by longer outputs rather than slower token generation.

6 Conclusion

The experiments show that sampling temperature mainly affects the number of tokens generated by the model. Low and medium temperatures produce similar response times and output lengths, while high temperatures generate longer answers and modestly increase latency. The generation time per token remains nearly constant, so the model is not slower per token at higher temperature. throughput gains.

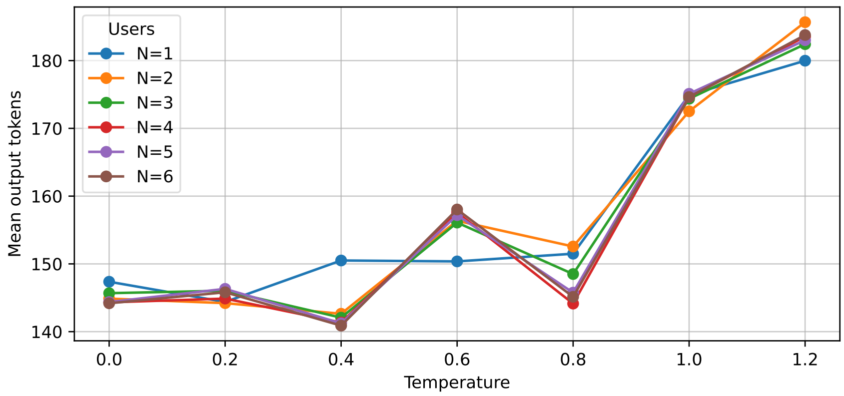

Fig. 9. Mean output tokens versus temperature for each number of users.

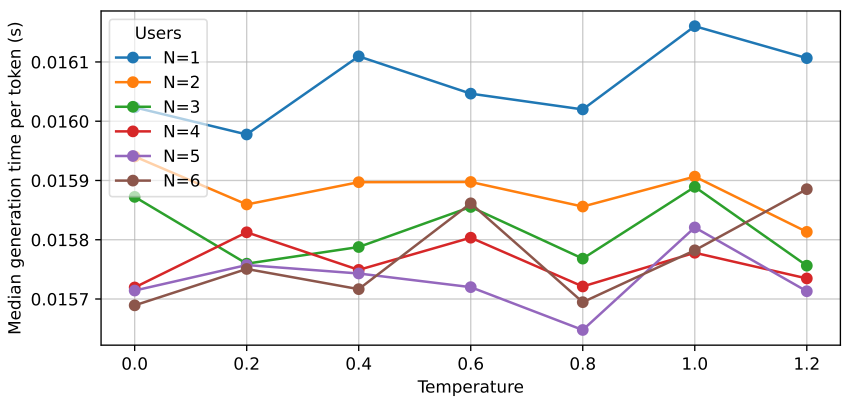

Fig. 10. Median generation time per token versus temperature for each number of users.